

C++ friend Function and friend Classes

Concept

- Data hiding is a fundamental concept of object-oriented programming. It restricts the access of private members from outside of the class.
- Similarly, protected members can only be accessed by derived classes and are inaccessible from outside.

Friend Function

- Feature in C++ called **friend functions** that break private rule and allow us to access member functions from outside the class.
- A **friend function** can access the **private** and **protected** data of a class. We declare a friend function using the friend keyword inside the body of the class.

Friend Function

```
class className {  
    ... ..  
    friend returnType functionName(arguments);  
  
};
```

Example

```
#include <iostream>
```

```
class Distance {
```

```
    private:
```

```
        int meter;
```

```
        friend int addFive(Distance);
```

```
    public:
```

```
        Distance() : meter(0) {}
```

```
};
```

Example

```
int addFive(Distance d) {  
    d.meter += 5;  
    return d.meter;  
}  
  
int main() {  
    Distance D;  
    cout << "Distance: " << addFive(D);  
    return 0;  
}
```

Example

```
#include <iostream>

class ClassB;

class ClassA {
    public:
    ClassA() : numA(12) {}
    private:
    int numA;
    friend int add(ClassA, ClassB);
};
```

Example

```
class ClassB {  
    public:  
    ClassB() : numB(1) {}  
    private:  
    int numB;  
    friend int add(ClassA, ClassB);  
};
```


Example

```
int add(ClassA objectA, ClassB objectB) {  
    return (objectA.numA + objectB.numB);  
}  
  
int main() {  
    ClassA objectA;  
    ClassB objectB;  
    cout << "Sum: " << add(objectA, objectB);  
    return 0;  
}
```

friend Class in C++

```
class ClassB;  
class ClassA {  
    // ClassB is a friend class of ClassA  
    friend class ClassB;  
    ... ..  
}  
class ClassB {  
    ... ..  
}
```

Example

```
#include <iostream>

class ClassB;

class ClassA {
    private:
        int numA;
    friend class ClassB;
    public:
        ClassA() : numA(12) {}
};
```

example

```
class ClassB {  
    private:  
        int numB;  
    public:  
        ClassB() : numB(1) {}  
        int add() {  
            ClassA objectA;  
            return objectA.numA + numB;  
        }  
};
```

Example

```
int main() {  
    ClassB objectB;  
    cout << "Sum: " << objectB.add();  
    return 0;  
}
```

Access functions

- To allow clients to read the value of *private* data, the class can provide a *get* function.
- To allow clients to modify *private* data, the class can provide a *set* function.

```
#include <iostream.h>
```

```
class rectangle {  
private:  
    float height;  
    float width;  
    int xpos;  
    int ypos;
```

```
public:  
    rectangle(float, float); // constructor  
    void draw();           // draw member function  
    void posn(int, int);    // position member function  
    void move(int, int);    // move member function  
    float get_height();     // access function  
    void set_height(float); // access function  
};
```

```
float rectangle::get_height()
{
    return (height);
}

void rectangle::set_height(float h)
{
    height = h;
}

void main()
{
    rectangle rc(1.0, 3.0);
    float value;

    value = rc.get_height();
    cout << "height: " << value << endl;

    rc.set_height(10.0);
    value = rc.get_height();
    cout << "height: " << value << endl;
}
```

Friend functions

- Friend functions of an object can "see" inside the object and access private member functions and data.
- To declare a function as a friend of a class, precede the function prototype in the class definition with the keyword *friend*.


```
class rectangle {  
  
    friend void set_value(rectangle&, float); // friend declaration  
  
private:  
    float height;  
    float width;  
    int xpos;  
    int ypos;  
public:  
    rectangle(float, float); // constructor  
    void draw();             // draw member function  
    void posn(int, int);      // position member function  
    void move(int, int);      // move member function  
    float get_height();       // access function  
    void set_height(float);   // access function  
};
```

```
void set_value(rectangle &rc, float h)
{
    rc.height = h;
}
```

```
void main()
{
    rectangle rc(1.0, 3.0);

    set_value(rc, 10.0);
}
```